

Verteile Systeme

14. Juni 2026

1 Aufgabe 1: Ein Feuerwerk an UDP-Nachrichten (Pseudo-Verteilt)

1.1 Implementierung

Für die Implementierung der Aufgabe wurde Python verwendet. Zunächst werden die Sockets initialisiert. Das ist sozusagen das “Adressbuch-Setup” für jeden Prozess. Dabei gibt es zwei separate Sockets für die beiden unterschiedlichen Kommunikationsmuster:

- Token

Listing 1: Auszug aus ring_process.py

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("127.0.0.1", my_port))
```

Beim Tokensocket handelt es sich um die Adresse von jedem Prozess über welche das Token weitergegeben wird.

- Token

1.1.1 Multicast: Feuerwerksocket

Listing 2: Auszug aus ring_process.py

```
mcast_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
mcast_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
mcast_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEPORT, 1)
mcast_sock.bind("", MCAST_PORT)
```

Das ist der zweite, separate UDP-Socket. Dieser ist nötig, weil dieser Socket einer Multicast-Gruppe beitreten soll.

Die Anfangswahrscheinlichkeit, dass ein Streichholz zündet wird initial auf 0.5 gesetzt. Bevor das erste Token von Prozess null losgeschickt wird, muss jeder Prozess starten und mit Ready an den Base-Port signalisieren bereit zu sein. Andernfalls kann es passieren, dass das Prozess null das Token weiterschickt und ein Folgeprozess noch gar nicht bereit ist wodurch das Token verloren geht.

Dann folgt die Kernlogik des Programms: In der while-Schleife warten zunächst alle Prozesse bis sie entweder ein Token oder ein Multicast erhalten. Erhält ein Prozess ein Token, wird das Token an den nächsten Prozess weitergeleitet. Je nachdem ob zufällig das Streichholz zünden soll, zündet es auch bevor weiterreichung und es wird eine Nachricht an alle Prozesse gesendet.

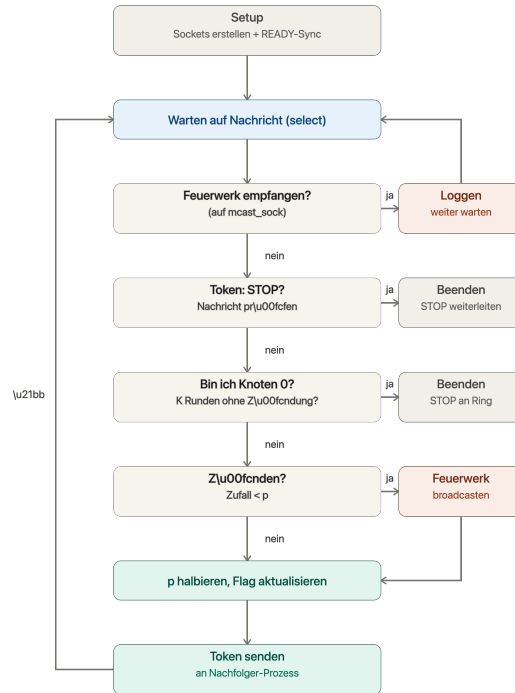


Abbildung 1: Ablauf eines Prozesses im Token-Ring-Feuerwerk-Algorithmus.

1.2 Ergebnisse

Das höchste n welches ich einigermaßen zuverlässig verwenden kann ist $n=256$. In manchen Anläufen klappt sogar $n=1024$. Bei $n=2048$ kommt es regelmäßig zu einem Timeout. Grund hierfür könnte sein, dass das Token verloren geht oder mein Computer die Last nicht tragen kann. Mit der Zeile

Listing 3: Auszug aus `run_experiments.py`

```
all_round_times = times[1:] if len(times) > 1 else times
```

wird bewusst die erste Runde direkt nach der READY-Synchronisation verworfen da sonst das Gesamtergebnis durch Aufwärm-Effekte verzerrt wäre.

n	Runden	Multicasts	min. Rundenzeit (ms)	mittl. Rundenzeit (ms)	max. Rundenzeit (ms)
2	5	2	0.077	0.099	0.149
4	4	5	0.196	0.307	0.523
8	7	12	0.372	0.799	1.556
16	6	15	0.710	1.219	2.280
32	8	32	1.427	3.571	9.283
64	7	60	2.469	8.226	18.183
128	9	118	7.806	24.009	62.763
256	12	253	14.072	67.192	308.490
512			Timeout (30 s)		
1024	15	1020	44.918	634.394	4188.559
2048			Timeout (30 s)		

Tabelle 1: Ergebnisse Aufgabe 1: Token-Runden, Gesamtzahl Multicasts (Feuerwerke) und Rundenzeiten in Abhängigkeit von n .

Eine interessante Beobachtung ist, dass die Ergebnisse nicht stabil reproduzierbar sind. In einigen Durchläufen war $n=512$ möglich in anderen nicht. Auf diese Beobachtung gehe ich nochmal in Aufgabe vier ein.

2 Aufgabe 2 – Ein Feuerwerk an UDP-Nachrichten (Verteilt)

Da mir leider keine weiteren Rechner zur Verfügung standen, habe ich um Aufgabe zwei zu lösen eine Virtual Machine aufgesetzt. Ergebnis:

Listing 4: Auszug aus `run_experiments.py`

```
STATS fires=0
STATS received=1
STATS rounds=3 round_times=[0.0006470680236816406, 0.0003540515899658203, 0.0
```

3 Aufgabe 3 – Ein simuliertes Feuerwerk

3.1 Implementierung

Aufgabe 3 wurde mit `sim4da` umgesetzt, ausgehend von `OneRingToRuleThemAllTest`. Drei Nachrichtentypen sind als `records` definiert, die das Interface `Message` implementieren:

```
record Token(boolean firedThisRound) implements Message {}
record Firework(String from) implements Message {}
record Stop() implements Message {}
```

`Token` entspricht `TOKEN:0/TOKEN:1` aus Aufgabe 1, `Firework` der Multicast-Nachricht, `Stop` dem `STOP`-Paket.

Die Logik steckt in `RingNode` (erbt von `Node`). Die Felder entsprechen den globalen Variablen aus `ring_process.py`: `p`, `fireCount`, `noFireCounter`, `lastSendTime`, `roundTimesNanos`, plus Identitätsfelder `id`, `nextId`, `isCoordinator`, `k`.

Die Methode `engage()` entspricht der `while True`-Schleife. Statt `select()` über zwei Sockets gibt es eine gemeinsame Mailbox, ausgewertet per `switch`:

```
switch (rm.message()) {
    case Token(boolean firedThisRound) -> { ... }
    case Firework f -> { ... }
    case Stop s -> { ... }
}
```

Zwei Teile aus Aufgabe 1 entfallen vollständig: das Multicast-Socket-Setup (`IP_ADD_MEMBERSHIP`, `SO_REUSEPORT`) – ersetzt durch `broadcast(new Firework(...))` – und die `READY`-Synchronisation, da `simulator.simulate()` alle Knoten gleichzeitig startet.

Der Algorithmus selbst (Halbierung von `p`, Zündung mit Wahrscheinlichkeit `p`, Flag-Weiterreichung, `K`-Runden-Terminierung) wurde unverändert übernommen. Tabelle ?? zeigt die Entsprechungen im Detail.

Für die Experimente wurde die Logik in `runSimulation(n, k, verbose)` ausgelagert, die ein `RunResult` (Zündungen, Runden, min/mittl./max. Rundenzeit, Laufzeit) zurückgibt. Rundenzeiten werden wie in Aufgabe 1 mit echter Wanduhrzeit (`System.nanoTime()`) gemessen.

3.2 Ergebnisse

Tabelle 2 zeigt die Ergebnisse für $n \in \{2, 4, 8, \dots, 32768\}$ bei $K = 3$ und $p_0 = 0,5$.

n	totalFires	Runden	min. (ms)	mittl. (ms)	max. (ms)	wallTime (ms)
2	3	6	0.086	1.619	9.162	10
4	5	5	0.189	0.302	0.498	1
8	10	6	0.442	0.566	0.828	4
16	15	7	0.633	0.952	2.173	7
32	22	12	0.615	1.154	3.041	15
64	63	8	0.725	2.891	9.712	25
128	121	12	1.507	5.777	27.154	73
256	273	11	1.539	13.920	75.044	157
512	539	11	1.747	20.323	117.971	228
1024	1085	13	2.155	43.061	269.174	565
2048	2078	14	3.189	122.296	860.918	1721
4096	4181	16	7.339	442.916	3445.186	7103
8192	8217	16	19.755	1854.012	15010.528	29704
16384	16269	16	38.565	7711.983	63234.528	123466
32768	32663	18	77.357	28699.614	266950.930	516960

Tabelle 2: Ergebnisse Aufgabe 3: Gesamtzahl Zündungen, Anzahl Runden und Rundenzeiten in Abhängigkeit von n .

Drei Beobachtungen fallen auf:

Erstens gilt für größere n näherungsweise $\text{totalFires} \approx n$ (z. B. $n = 16384 \rightarrow 16269$, $n = 32768 \rightarrow 32663$). Das entspricht der Erwartung, da jeder Prozess über die gesamte Laufzeit mit Wahrscheinlichkeit $0,5 + 0,25 + 0,125 + \dots \approx 1$ mindestens einmal zündet.

Zweitens bleibt die Anzahl der Runden mit Werten zwischen 5 und 18 nahezu unabhängig von n , da p bereits nach wenigen Runden gegen 0 konvergiert und das K-Runden-Kriterium dann schnell erreicht wird.

Drittens wächst `wallTimeMs` deutlich überlinear. Bei jeder Verdopplung von n steigt die Laufzeit um einen Faktor, der für große n gegen $\approx 4,2$ konvergiert (z. B. $8192 \rightarrow 16384$: Faktor 4,16; $16384 \rightarrow 32768$: Faktor 4,19). Da $4 \approx 2^2$, deutet dies auf ein quadratisches Wachstum $O(n^2)$ hin: Insgesamt feuern $\approx n$ Knoten je einen `broadcast()`, der jeweils an $n - 1$ andere Knoten zugestellt wird, sodass die Gesamtzahl der ausgetauschten Nachrichten in der Größenordnung n^2 liegt.

3.2.1 Maximal erreichbares n

Als Abbruchkriterium wurde eine praktische Laufzeitgrenze von 5 Minuten pro Lauf definiert. $n = 16384$ benötigte 123s und liegt damit noch innerhalb dieser Grenze, $n = 32768$ benötigte bereits 517s und überschreitet sie. Damit ergibt sich $n_{\max} = 16384$.

Im Gegensatz zu Aufgabe 1 handelt es sich hierbei nicht um einen Absturz (keine Exception, kein `OutOfMemoryError`), sondern ausschließlich um die praktische Laufzeit, die durch das quadratische Nachrichtenwachstum bedingt ist.

4 Aufgabe 4 – Konsistenz

4.1 Mögliche Inkonsistenzen

In der vorliegenden Anwendung sind zwei Stellen denkbar, an denen verschiedene Prozesse eine unterschiedliche Sicht auf den Programmablauf haben könnten: Erstens könnte das Token verloren gehen oder dupliziert werden, sodass einzelne Knoten unterschiedliche Vorstellungen davon

hätten, in welcher Runde sich das Protokoll gerade befindet. Zweitens könnte eine **Firework**-Nachricht nicht bei allen Empfängern ankommen, sodass Knoten unterschiedliche Sichten darauf hätten, wie viele Zündungen insgesamt stattgefunden haben. Bei Aufgabe 1/2 (UDP) sind beide Fälle prinzipiell möglich (Paketverlust, Duplikation); in `sim4da` garantieren `send()`/`broadcast()` dagegen eine zuverlässige Zustellung.

Daraus ergeben sich zwei prüfbare Kriterien:

- **Rundenkonsistenz:** Jedes **Token** trägt eine Rundenummer **round**. Jeder Knoten erwartet, dass diese beim Empfang genau eins höher ist als die zuletzt gesehene Rundenummer. Abweichungen würden auf Token-Verlust oder -Duplikation hindeuten.
- **Broadcast-Vollständigkeit:** Jede gesendete **Firework**-Nachricht muss bei genau $n - 1$ anderen Knoten ankommen. Es muss also gelten: $\sum(\text{empfangene Fireworks}) = \text{totalFires} \times (n - 1)$.

4.2 Implementierung der Erkennung

Beide Kriterien wurden in `RingFeuerwerkKonsistenz.java` als Erweiterung von Aufgabe 3 umgesetzt. **Token** und **Firework** wurden um das Feld **round** ergänzt; `Token(false, 1)` startet die Simulation. Jeder `RingNode` führt einen lokalen Zähler `localRound` und prüft beim Empfang `round == localRound + 1`; bei Abweichung wird `roundViolation = true` gesetzt und eine Meldung ausgegeben. Zusätzlich zählt jeder Knoten empfangene **Firework**-Nachrichten in `receivedFireworks`. Nach Abschluss der Simulation werden beide Kriterien global ausgewertet und als **CONSISTENCY**-Zeile ausgegeben.

4.3 Ergebnisse

Für $n = 5$ ergab sich:

```
CONSISTENCY n=5 round=OK broadcast=OK (received=28 expected=28)
```

Beide Kriterien waren erfüllt: kein Knoten meldete eine abweichende Rundenummer, und die Anzahl empfangener Fireworks (28) entsprach exakt $\text{totalFires} \times (n - 1) = 7 \times 4 = 28$.

Dieses Ergebnis war erwartbar, da `send()`/`broadcast()` in `sim4da` zuverlässig sind – beide Kriterien sind hier durch die Konstruktion des Frameworks garantiert, nicht nur empirisch erfüllt. Interessanterweise ergab auch der Testlauf von Aufgabe 2 ($n=2$, über Host und VM via UDP) ein konsistentes Ergebnis ($\text{fires} = 1$, $\text{received} = 1 = 1 \times (2 - 1)$). Im Gegensatz zu `sim4da` ist dies bei UDP jedoch keine garantierte Eigenschaft, sondern lediglich das Ergebnis dieses einzelnen Laufs – bei Paketverlust könnte das Kriterium verletzt werden, ohne dass die Implementierung dies erkennt.